



CertForge

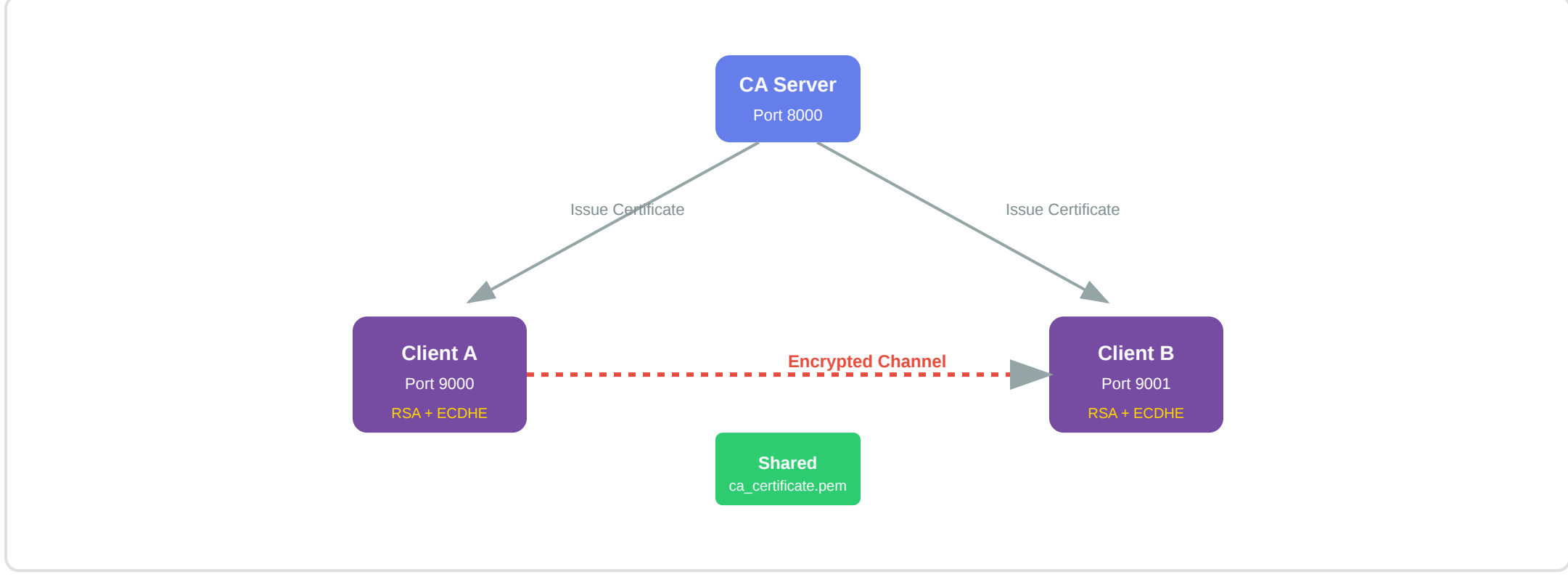
Authenticated Key Exchange & Encrypted Communication(Without TLS)

Practical Lab Assignment
Information Security

Group Members:
1) Anand Homkumare (2025PCS0019)
2) Hussain Ahmad (2025PCS0029)
Institution: IIT Jammu

1 / 8

System Architecture



Certificate Authority

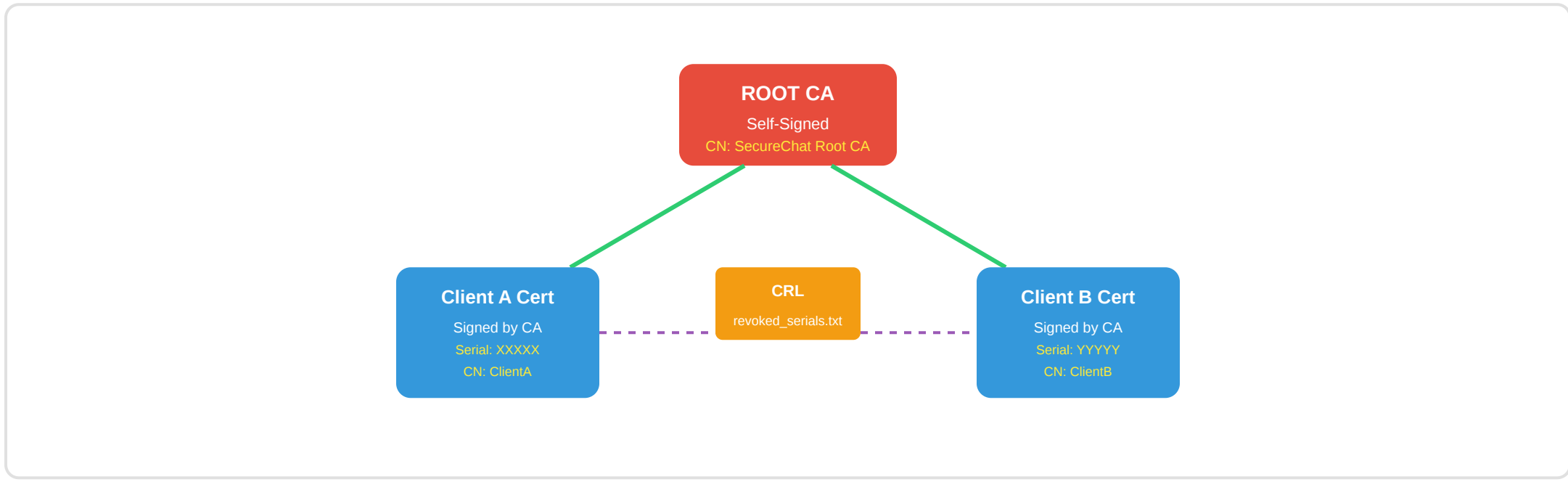
- Root CA with self-signed cert
- Issues client certificates
- Maintains revocation list
- Password-protected private key

Chat Clients

- RSA-2048 identity keys
- Request certs from CA
- ECDHE for session keys
- AES-GCM encryption

2 / 8

PKI & Trust Chain



Certificate Validation Checks:

- CA Signature Verification (RSA-PKCS1v15)
- Valid Time Window
- Expected Identity Match
- Not Revoked (CRL Check)
- Not a CA Certificate
- Basic Constraints Check

3 / 8

Authenticated ECDHE Key Exchange

1. Ephemeral ECDH: Generate secp256r1 key pair (EC_A, EC_A)

↓

2. RSA Signature: Sign ephemeral pub key with RSA identity key (Sign(EC_A, RSA_priv))

↓

3. Exchange: Send (EC_A || Sig_A) → Receive (EC_B || Sig_B)

↓

4. Verify: Check signature using peer's RSA public key from certificate

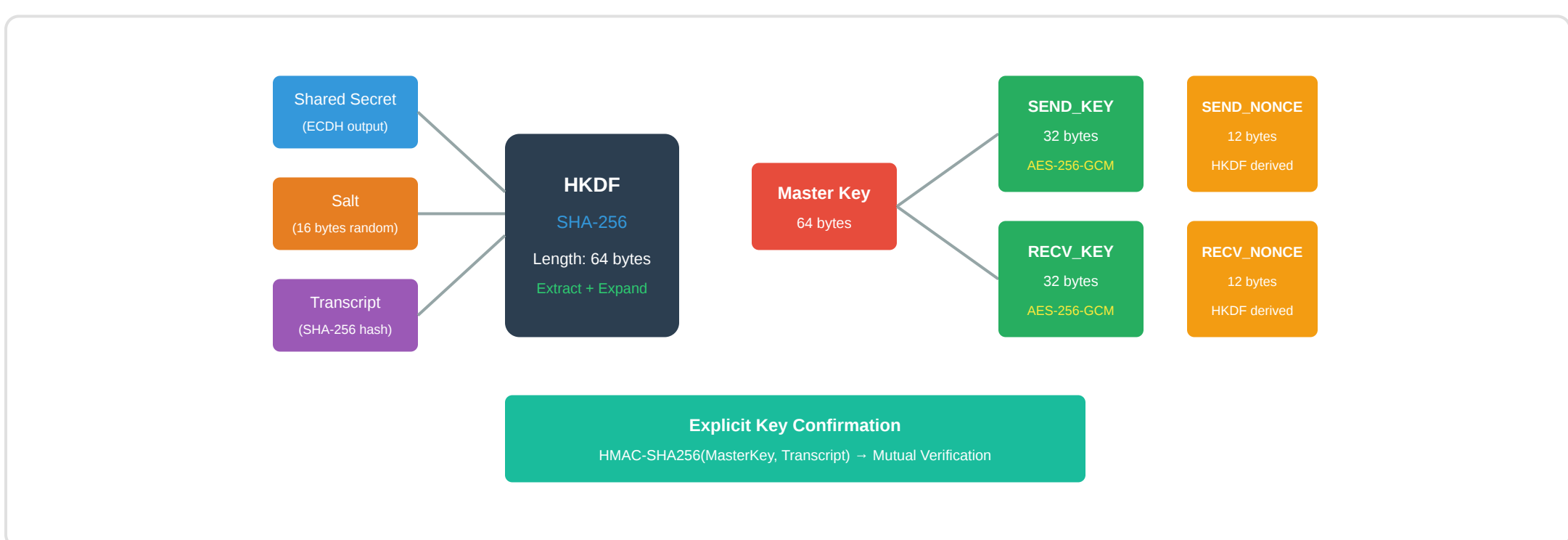
↓

5. Shared Secret: Compute (SS = ECDH(ec_Ay, EC_B))

Security Property: Ephemeral keys provide *Forward Secrecy* — compromise of long-term RSA keys cannot decrypt past sessions.

4 / 8

Key Derivation & Session Establishment



Directional Keys

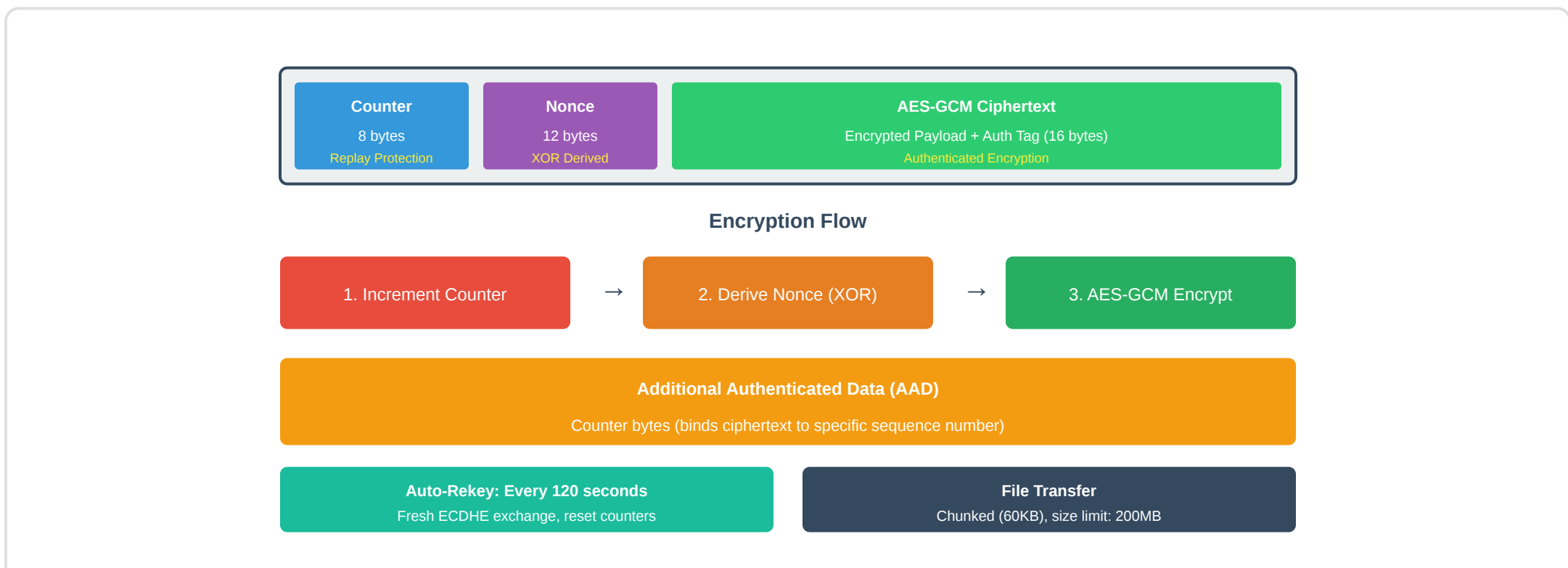
Client A: SEND = bytes 0-31, RECV = bytes 32-63
Client B: RECV = bytes 0-31, SEND = bytes 32-63
Prevents reflection attacks

Nonce Generation

Per-message nonce =
Nonce_Base XOR Counter

5 / 8

Secure Data Transmission



Replay Protection

Strict counter ordering rejects old messages



Authenticated Encryption

AES-256-GCM provides confidentiality + integrity



Forward Secrecy

Periodic rekeying limits key exposure window



Secure File Transfer

Chunked transmission with size validation

6 / 8

Security Features & Threat Mitigation

Threat	Mechanism	Implementation
Man-in-the-Middle	Certificate Validation	CA-signed certs, RSA signature verification
Replay Attacks	Sequence Counters	64-bit strict monotonic counters per direction
Key Compromise	Forward Secrecy	Ephemeral ECDHE, periodic rekeying (120s)
Message Tampering	Authenticated Encryption	AES-256-GCM with AAD binding
Identity Spoofing	PKI + Explicit Confirmation	Certificate chain + HMAC key confirmation
Reflection Attacks	Directional Keys	Send/Recv key separation per role
Revoked Certificates	CRL Checking	Local revoked_serials.txt validation

Protocol Strengths:

- Custom protocol (not TLS) meeting assignment requirements
- Hybrid cryptography: RSA for auth, ECDH for secrecy
- Defense in depth: Multiple overlapping security layers
- Minimal trust: Clients verify everything independently

7 / 8

Implementation Overview

Project Structure

```
secure_chat_project/  
├── ca_server/  
│   ├── ca_server.py  
│   ├── ca_private.key.pem  
│   ├── ca_certificate.pem  
│   └── revoked_serials.txt  
├── clientA/  
│   ├── clientA.py  
│   ├── A_key.pem  
│   └── A_cert.pem  
├── clientB/  
│   ├── clientB.py  
│   ├── B_key.pem  
│   └── B_cert.pem  
└── shared/  
    └── ca_certificate.pem
```

Key Functions

- generate_key() - RSA-2048 identity
- create_csr() - Certificate request
- secure_key_exchange() - ECDHE + HKDF
- verify_certificate() - Full chain validation
- send_file() - Chunked encrypted transfer

Execution Flow

- Start CA Server:
python ca_server.py
- Start Client A:
python clientA.py
Auto-generates keys, requests cert
- Start Client B:
python clientB.py
Same initialization process
- Connect:
Select "Connect to Peer" in menu
🔑 Secure channel established!

Commands:

/sendfile <path> - Transfer file

Thank You!

8 / 8